Design Pattern Builder

Problème 1

```java
public class House {
    private String foundation;
    private String structure;
    private String roof;
    private String interior;
    private Boolean garage;
    private Boolean pool;
    private Boolean garden;
    private String fence;
    private Boolean security;

    public House(String foundation, String structure, String roof,
                 String interior, Boolean garage, Boolean pool,
                 Boolean garden, String fence, Boolean security) {
        this.foundation = foundation;
        this.structure = structure;
        this.roof = roof;
        this.interior = interior;
        this.garage = garage;
        this.pool = pool;
        this.garden = garden;
        this.fence = fence;
        this.security = security;
    }
}
```

```java
House house = new House("concrete", "brick", "tiles", null, true, false, true, null, false);
// Que signifie ce true ? Ce false ?
```

Attributs et paramètres de constructeur à rallonge

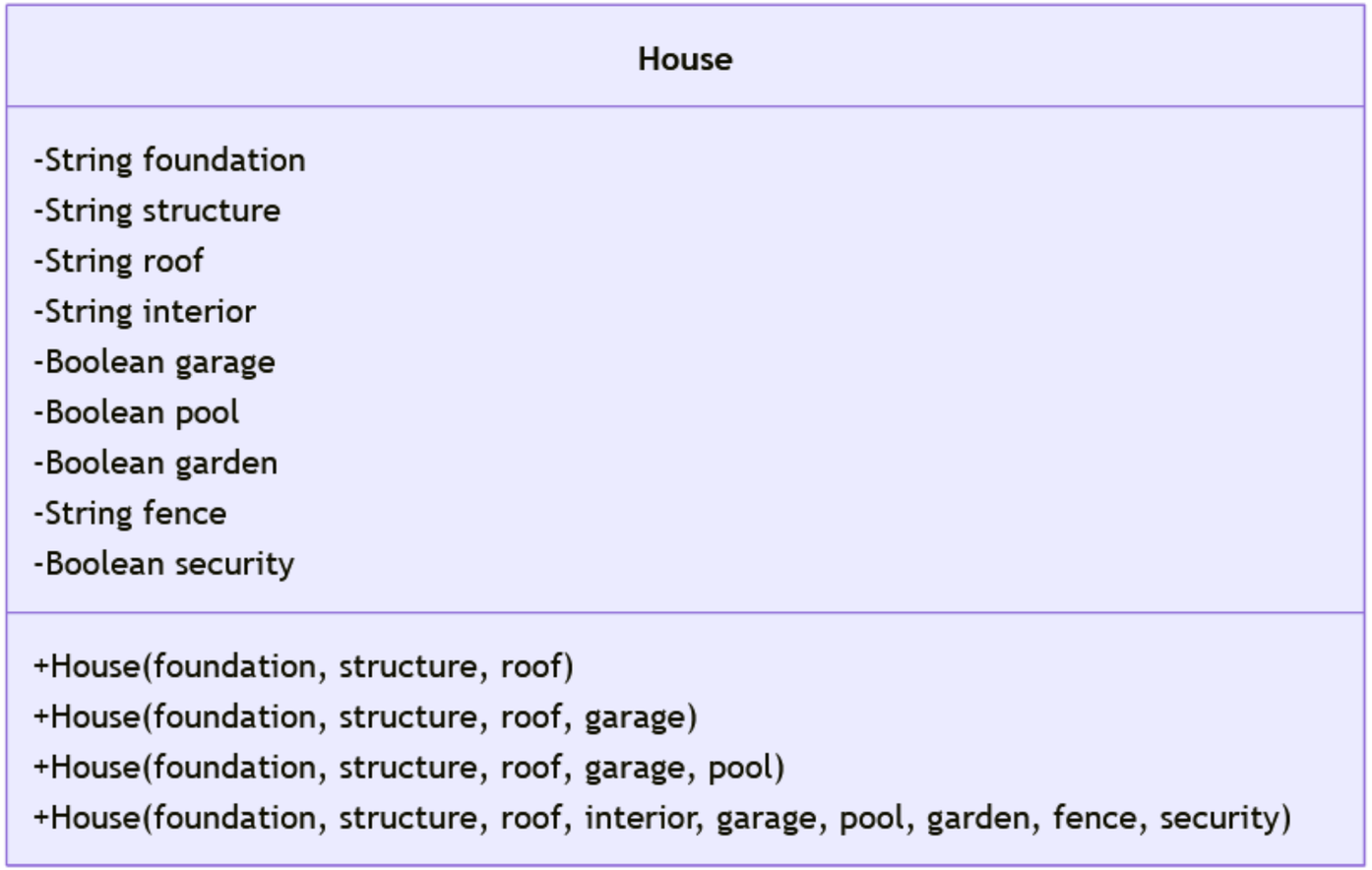
Extrêmement difficile de définir une nouvelle maison

Problème 2

```java
public class House {
    public House(String foundation, String structure, String roof) {
        // Constructeur de base
    }

    public House(String foundation, String structure, String roof, boolean garage) {
        // Avec garage
    }

    public House(String foundation, String structure, String roof, boolean garage, boolean pool) {
        // Avec garage et piscine
    }

    // ... explosion combinatoire !
}
```

- Explosion de nombre de constructeurs
- Code dupliqué
- Maintenance cauchemardesque

UML du problème



Solution

La classe House ne possède que ses attributs (et leurs getters au besoin).
Constructeur vide car tout va se construire pas-à-pas.

```java
public class House {
    private String foundation;
    private String structure;
    private String roof;
    private String interior;
    private boolean garage;
    private boolean pool;
    private boolean garden;

    // Constructeur privé ou package-private
    House() {
    }
```

Solution

Création d'une classe Builder qui utilise des setters pour définir les attributs de la classe principale

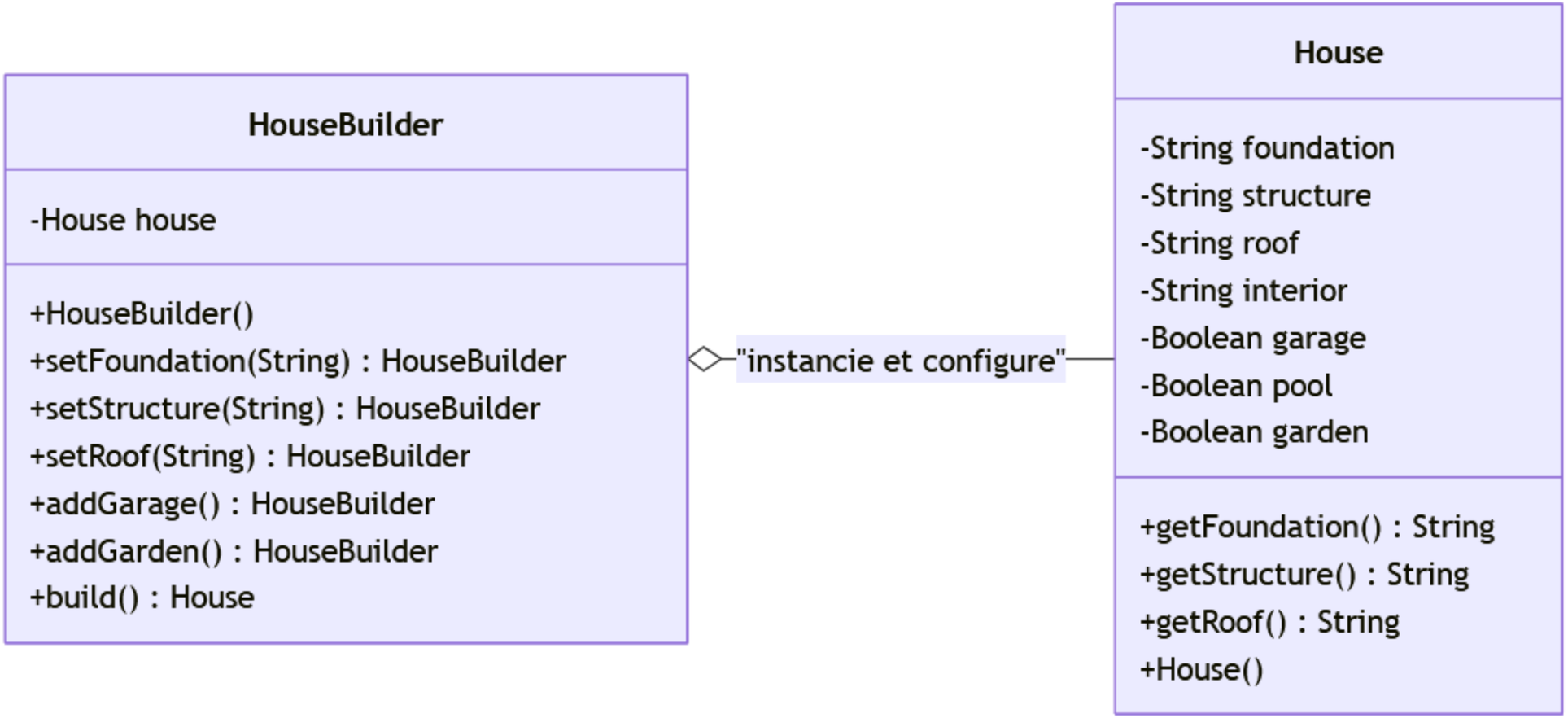
```java
public class HouseBuilder {
    private House house;

    public HouseBuilder() {
        this.house = new House();
    }

    public HouseBuilder setFoundation(String foundation) {
        house.foundation = foundation;
        return this;  // Retourne this pour chaînage
    }

    public HouseBuilder setStructure(String structure) {
        house.structure = structure;
        return this;
    }

    public HouseBuilder setRoof(String roof) {
        house.roof = roof;
        return this;
    }
```

Solution

Instantiation de l’objet House et utilisation du Builder

```
House house = new HouseBuilder()
        .setFoundation("concrete")
        .setStructure("brick")
        .setRoof("tiles")
        .addGarage()
        .addGarden()
        .build();
```

UML Solution



Pourquoi c’est un pattern créationnel ?

- Création d’objets
- Flexible, réutilisable

Les 5 patterns créationnels du GoF:

- **Singleton**
- **Factory Method**
- **Abstract Factory**
- **Prototype**
- ***Builder***

Caractéristiques du Builder

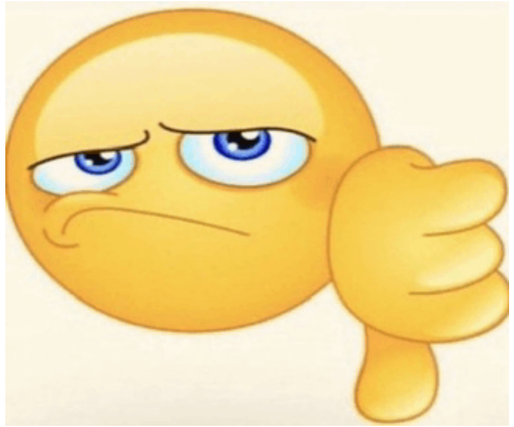
- Encapsule logique de création
- Contrôle processus d’instanciation
- Offre interface de création flexible
- Sépare construction de réprésentation

Quand utiliser Builder ?

Utiliser Builder quand:

- Beaucoup de paramètres
- Paramètres optionnels
- Construction plusieurs étapes
- Différentes représentations

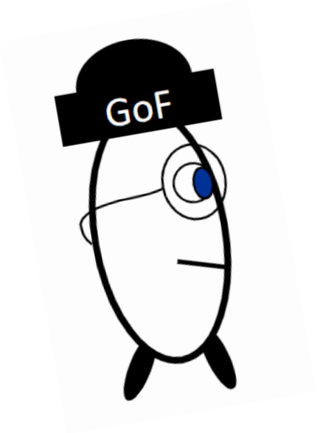
etc.

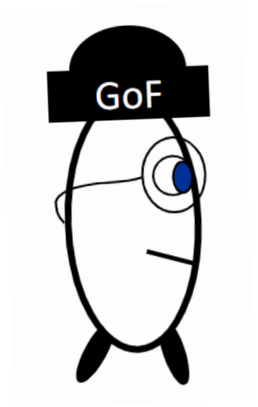


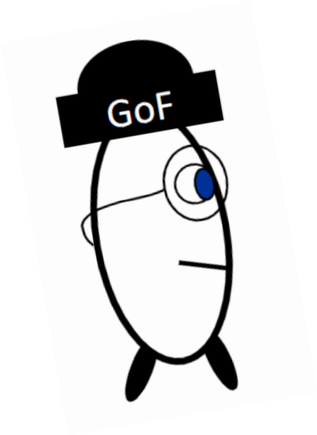
Ne PAS utiliser Builder quand:

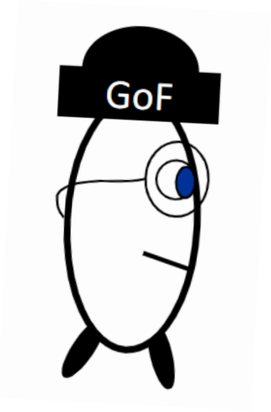
- Objet simple
- Paramètres obligatoires
- Construction triviale

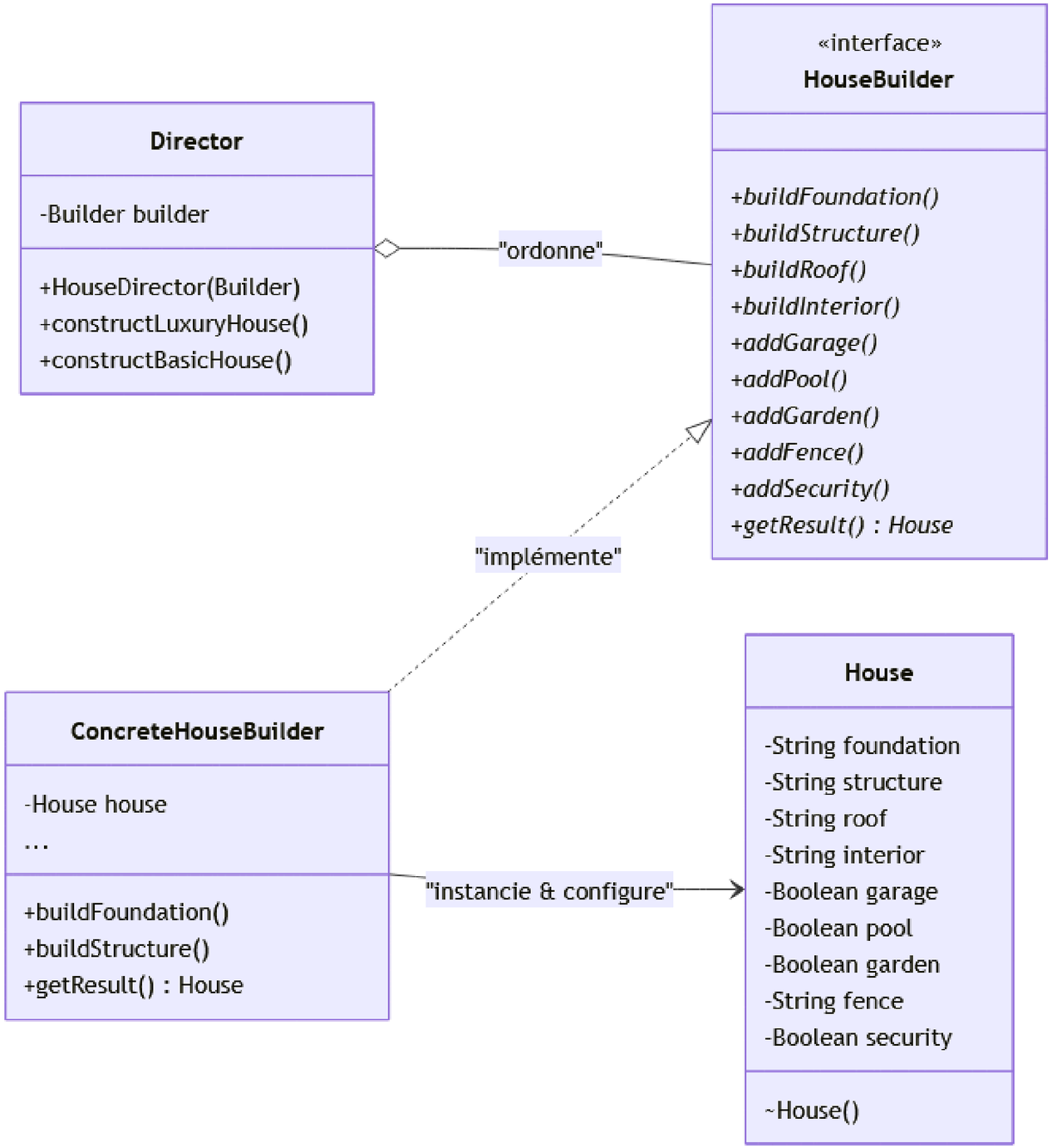
Exemple GoF complet











Exemple GoF complet 1/5

```java
// Le produit
public class House {
    private String foundation;
    private String structure;
    private String roof;
    private String interior;
    private boolean garage;
    private boolean pool;
    private boolean garden;
    private String fence;
    private boolean security;

    // Constructeur package-private pour forcer l'usage du Builder
    House() {}

    // Setters utilisés par le Builder
    void setFoundation(String f) { this.foundation = f; }
    void setStructure(String s) { this.structure = s; }
    void setRoof(String r) { this.roof = r; }
    void setInterior(String i) { this.interior = i; }
    void setGarage(boolean g) { this.garage = g; }
    void setPool(boolean p) { this.pool = p; }
    void setGarden(boolean g) { this.garden = g; }
    void setFence(String f) { this.fence = f; }
    void setSecurity(boolean s) { this.security = s; }
```

C’est le produit que l’on souhaite.

- Contient tous les attributs
- Constructeur vide et en inaccessible pour l’utilisateur : seul le builder créera l’objet

Exemple GoF complet 2/5

```java
public interface HouseBuilder {
    void buildFoundation();
    void buildStructure();
    void buildRoof();
    void buildInterior();
    void addGarage();
    void addPool();
    void addGarden();
    void addFence();
    void addSecurity();
    House getResult(); // La méthode "getResult" du schéma
}
```

Interface : déclare les méthodes des classes qui l’implémenteront

Exemple GoF complet 3/5

Une classe qui implémente l'interface builder avec des matériaux précis:

Fondations en béton, Structure en briques, Toit en tuiles...

```java
public class ConcreteHouseBuilder implements HouseBuilder {
    private House house = new House();

    @Override public void buildFoundation() { house.setFoundation("Béton"); }
    @Override public void buildStructure() { house.setStructure("Briques"); }
    @Override public void buildRoof() { house.setRoof("Tuiles"); }
    @Override public void buildInterior() { house.setInterior("Moderne"); }
    @Override public void addGarage() { house.setGarage(true); }
    @Override public void addPool() { house.setPool(true); }
    @Override public void addGarden() { house.setGarden(true); }
    @Override public void addFence() { house.setFence("Bois"); }
    @Override public void addSecurity() { house.setSecurity(true); }

    @Override
    public House getResult() {
        House result = this.house;
        this.house = new House(); // Réinitialiser pour une nouvelle construction
        return result;
    }
}
```

```java
public class HouseDirector {
    private HouseBuilder builder;

    public HouseDirector(HouseBuilder builder) {
        this.builder = builder;
    }

    // Deux templates de construction pour différentes "recettes" de maison
    public void constructLuxuryHouse() {
        builder.buildFoundation();
        builder.buildStructure();
        builder.buildRoof();
        builder.addGarage();
        builder.addPool();
        builder.addSecurity(); // Recette spécifique
    }

    public void constructBasicHouse() {
        builder.buildFoundation();
        builder.buildStructure();
        builder.buildRoof();
        // Pas + d'options supplémentaires car basique
    }
}
```

Exemple GoF complet 4/5

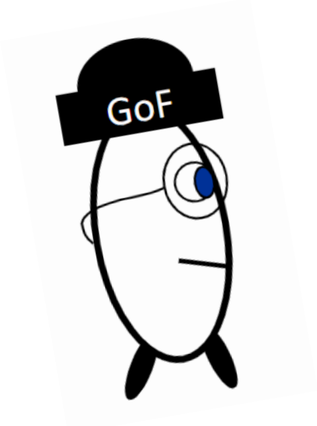
Simplification, Fiabilité et Réutilisabilité.

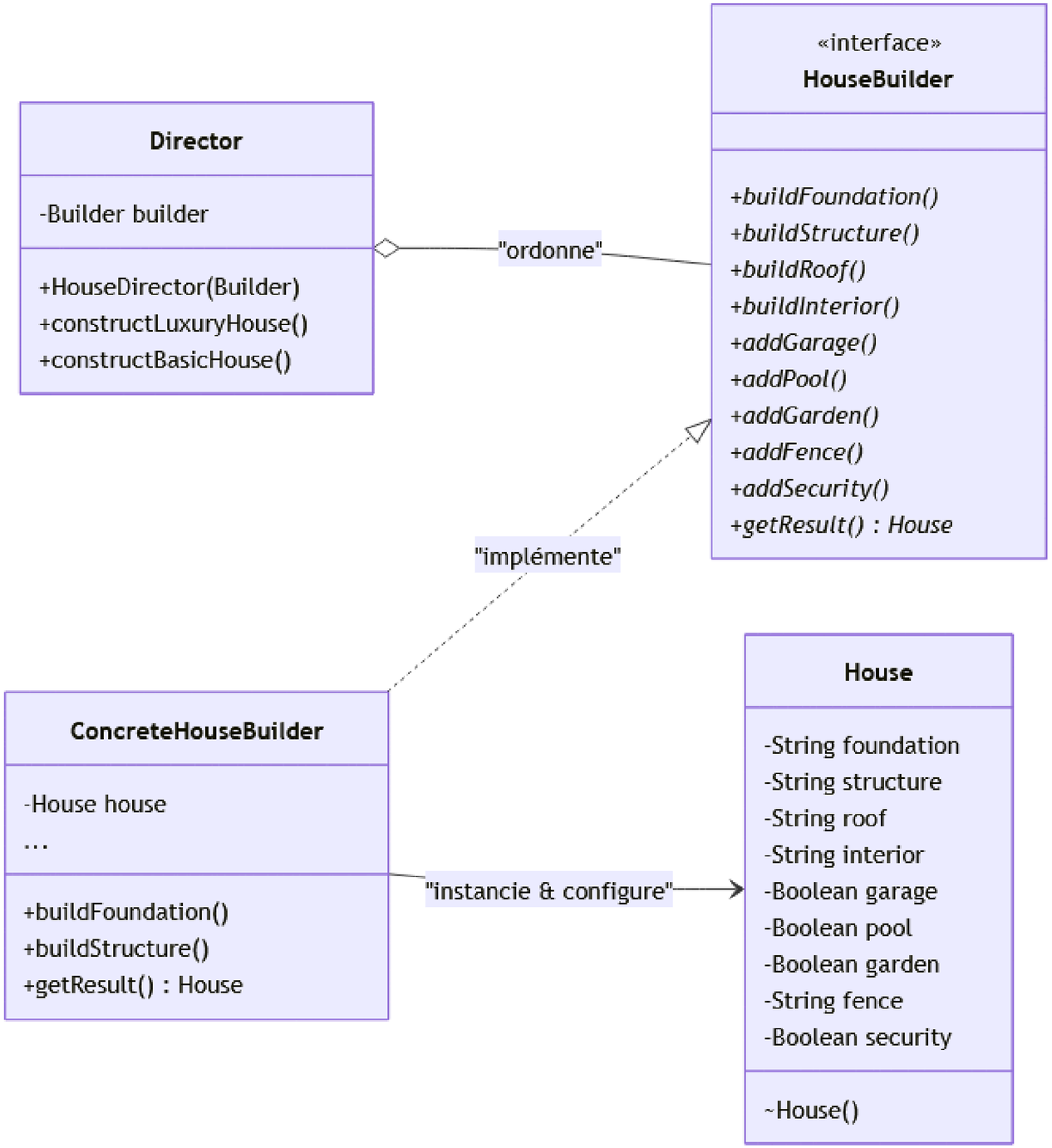
Création de plusieurs recettes permettant à l’utilisateur de ne pas écrire toutes les fonctions si une recette lui convient.

Exemple GoF complet 5/5

```java
HouseBuilder builder = new ConcreteHouseBuilder();

// Si l'utilisateur veut choisir lui-même ses options étape par étape
// plus besoin d'utiliser le director
builder.buildFoundation();
builder.buildStructure();
builder.addGarden();
builder.addPool();

House customHouse = builder.getResult();
System.out.println("Maison personnalisée : " + customHouse);
```

```
Maison basique construite : House [foundation=Béton, structure=Briques, pool=false, security=false]
```


GoF


Director
-Builder builder
+HouseDirector(Builder)
+constructLuxuryHouse()
+constructBasicHouse()
"ordonne"
«interface»
HouseBuilder
+buildFoundation()
+buildStructure()
+buildRoof()
+buildInterior()
+addGarage()
+addPool()
+addGarden()
+addFence()
+addSecurity()
+getResult() : House
"implémente"
ConcreteHouseBuilder
-House house
...
+buildFoundation()
+buildStructure()
+getResult() : House
"instancie & configure"
House
-String foundation
-String structure
-String roof
-String interior
-Boolean garage
-Boolean pool
-Boolean garden
-String fence
-Boolean security
~House()

TD - Builder pour un CV

Objectif: Créer un Builder pour générer un CV avec différentes sections.

Spécifications:

- Informations personnelles (obligatoire)
- Expériences professionnelles (liste)
- Formations (liste)
- Compétences (liste)
- Langues (liste)
- Centres d’intérêt (liste)

À faire:

1. Écrire les tests
2. Implémenter le builder
3. Valider que les sections obligatoires sont présentes

code initial disponible sur Teams